

System & Maintenance Documentation

This document covers the technical details of the **Ababil's Attire** project, providing a quick reference for developers or IT staff who may maintain the system in the future.

1. Tech Stack

- **Frontend Framework:** React 19 (via Vite)
- **Language:** TypeScript
- **Routing:** React Router v7
- **Database & Backend as a Service:** Supabase (PostgreSQL, Authentication, Storage, Row Level Security)
- **Styling:** Vanilla CSS (`index.css` & `App.css`)
- **Linting:** oxlint

2. Project Structure

- `src/components/` : Reusable UI components organized by `admin` and `customer` areas.
- `src/context/` : React context providers (e.g., Auth, Cart).
- `src/pages/` : Main route views, separated into `admin/` and `customer/` folders.
- `src/services/` : Supabase client wrappers, logic for invoices, database calls.
- `src/lib/` : Library utilities, like the initialized `supabase.ts` client.
- `supabase/migrations/` : SQL migration files defining tables and Row Level Security (RLS) policies.

3. Environment Variables

To run this project locally, a `.env` file is required in the project root:

```
``  
  
VITE_SUPABASE_URL=your_supabase_project_url  
  
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key
```

4. Supabase Setup

- **Authentication:** Admin users authenticate using Supabase Auth (Email/Password). Customer sessions (for tracking) rely on non-authenticated identifiers (Phone + Order ID).
- **Database Schema:** Stores products , orders , order_items , customers , and store_settings .
- **Row Level Security (RLS):** Enabled on all tables.
- Public can SELECT active products, INSERT orders, SELECT their own order (matched by phone/ID).
- Admin (authenticated user) can SELECT, INSERT, UPDATE, DELETE all tables.
- **Storage:** Product images are stored in a Supabase Storage bucket named product-images .

5. Known Limitations & Architecture Notes

- **bKash Integration:** There is no automated/API bKash integration. The checkout flow captures customer payment details (TrxID, Sender Number, Reference Name, Amount). Admin must manually reconcile these payments using their personal bKash app before fulfilling the order. This is a design decision to avoid complex merchant API approvals for a small business.
- **Customer Accounts:** There is no customer login system. Customers are identified by their phone numbers when tracking orders.

- **Admin Roles:** Any authenticated user via Supabase Auth is considered an "Admin". There is no granular role-based access control (RBAC) currently implemented.

6. Deployment & Backups

- The frontend can be deployed to static hosting platforms like Vercel, Netlify, or Render using `npm run build`.
- Supabase automatically handles database backups based on the project's pricing tier. Regular manual exports of the `orders` and `customers` tables via the Supabase dashboard are recommended.

7. Security Notes

- Never expose the Supabase `service_role` key in the frontend `.env`` file.
- Changes to database policies should be thoroughly tested to prevent exposing PII (Personally Identifiable Information) of customers.